

Final Exam Study Guide

CPSC 406 — Algorithm Analysis
Chapman University Spring 2026
Instructor: Jonathan Weinberger

This guide mirrors the structure of the final exam. For each problem, focus on the general method, not just the specific numbers in one practice example. A strong answer should show the main reasoning steps clearly.

Problem 1: Turing Machines

Core idea. A deterministic Turing machine follows exactly one transition from the current state and current tape symbol. To decide whether a word is accepted, simulate the machine until it enters either the accept state or the reject state.

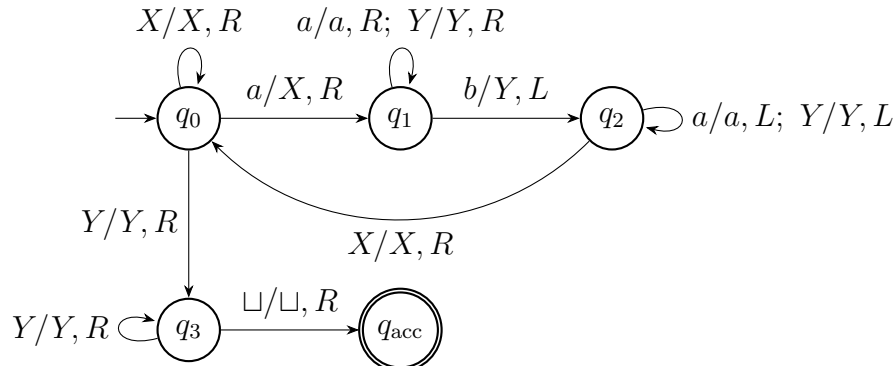
How to reason.

1. Start in the start state with the head on the first input symbol.
2. At each step, read the current state and tape symbol.
3. Apply the transition: write the new symbol, move left or right, and change state.
4. Stop only when the machine reaches a halting state.

Example machine. The following machine accepts exactly the language

$$\{a^n b^n : n \geq 1\}.$$

It repeatedly marks one unmatched a as X , finds one unmatched b and marks it as Y , and then returns left to look for the next unmatched a .



Omitted transitions would go to a rejecting state q_{rej} .

Example of correct reasoning. On input $aabb$, the machine changes the first a to X , changes the first unmatched b to Y , returns left, and repeats. It eventually sees only X 's followed by Y 's and accepts. The input abb is rejected because, after one match, an unmatched b remains where the machine expects either another a or only marked Y 's. The input $abab$ is also rejected because the symbols are not in the required block order.

Common pitfalls.

- Do not decide based only on the first few symbols. The machine's answer is determined by the halting state it eventually reaches.
- Do not ignore the blank symbol. A machine could accept or reject only after reading the blank at the end of the input.
- Do not assume every accepting state is reached immediately after seeing a key symbol. Often the machine must scan to the end first.

Problem 2: Landau Notation

Core idea. Big-O notation is about eventual upper bounds. To classify a sum of common terms, identify which term grows fastest as n becomes large.

Useful growth facts.

$$\log n \ll n^c \ll b^{\sqrt{n}} \ll a^n \quad \text{for constants } c > 0, a > 1, b > 1.$$

Subexponential terms such as $b^{\sqrt{n}}$ grow faster than every fixed polynomial but slower than ordinary exponentials such as a^n .

Example of correct reasoning. For

$$g(n) = 4n^5 + 9n^2 \log n + 7\sqrt{n},$$

the dominant term is n^5 . Therefore $g(n) \in \mathcal{O}(n^5)$ and $g(n) \in \mathcal{O}(n^6)$, but not $\mathcal{O}(n^4)$.

Common pitfalls.

- Big-O is an upper bound, not necessarily a tight bound. If $f(n) \in \mathcal{O}(n^2)$, then also $f(n) \in \mathcal{O}(n^3)$.
- Constants and lower-order terms do not determine asymptotic growth.
- Do not treat $b^{\sqrt{n}}$ like a polynomial. It eventually beats n^k for every fixed k .

Problem 3: CNF and Satisfiability

Core idea. A formula is in conjunctive normal form (CNF) when it is an AND of clauses, and each clause is an OR of literals. A literal is a variable or the negation of a variable.

How to convert to CNF.

1. Eliminate implications using $p \rightarrow q \equiv \neg p \vee q$.
2. Push negations inward using De Morgan's laws: $\neg(p \wedge q) \equiv \neg p \vee \neg q$ and $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
3. Distribute OR over AND when needed: $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$.

Example of correct reasoning.

$$(p \rightarrow (q \vee r)) \wedge \neg(q \wedge r) \equiv (\neg p \vee q \vee r) \wedge (\neg q \vee \neg r).$$

This formula is satisfiable; for instance, $p = \text{false}$, $q = \text{true}$, and $r = \text{false}$ make both clauses true.

Common pitfalls.

- Do not replace $p \rightarrow q$ with $p \vee q$. The correct form is $\neg p \vee q$.
- One failed truth assignment does not prove unsatisfiability.
- When checking CNF, every top-level connective should be an AND of clauses.

Problem 4: Logical Modeling and Resolution

Core idea. Translate each English statement into propositional logic, then convert the result into clauses. Resolution can prove inconsistency by deriving the empty clause $\square$.

Modeling patterns.

- “If P , then Q ” becomes $P \rightarrow Q$, equivalently $\neg P \vee Q$.
- “ P and Q ” becomes $P \wedge Q$.
- “ P must be blocked” may become $\neg U$ if U means the action is allowed or occurs.
- A reported fact becomes a unit clause, such as $\{L\}$ or $\{\neg A\}$.

Example of correct reasoning. Suppose M means “the printer is in maintenance mode” and P means “printing is allowed.” If a policy contains $M \rightarrow \neg P$ and a log contains both M and P , the clauses include

$$\{\neg M, \neg P\}, \quad \{M\}, \quad \{P\}.$$

Resolution gives

$$\{\neg M, \neg P\}, \{M\} \vdash \{\neg P\}, \quad \{\neg P\}, \{P\} \vdash \square.$$

Thus the policy and log are inconsistent.

Common pitfalls.

- Do not confuse an implication with its converse. $B \rightarrow U$ does not mean $U \rightarrow B$.
- Define variables before using them. Ambiguous variables lead to ambiguous formulas.
- In resolution, resolve only complementary literals such as U and $\neg U$.
- A single resolution step produces one resolvent from two clauses. If a proof needs several resolvents, write them as several steps; do not combine multiple eliminations into one resolution step. For example, do not write $\{\neg M, \neg P\}, \{M\}, \{P\} \vdash \square$ in one step; first derive $\{\neg P\}$, then resolve it with $\{P\}$.

Problem 5: Induction and Big-O Bounds

Core idea. To prove a recurrence is bounded by a proposed expression, use induction. The induction hypothesis should be substituted into the recursive term, then algebra should show the claimed bound.

Template.

1. State the claim.
2. Prove the base case directly.
3. Assume the claim for smaller input, namely all $k < n$ (e.g. $k = \frac{n}{2}$ or $k = n - 1$).
4. Use the recurrence and substitute the induction hypothesis.
5. Finish the algebra and state the conclusion.

Example of correct reasoning. If $T(1) = 1$ and $T(n) \leq 2T(\frac{n}{2}) + n$ for powers of two, a natural claim is $T(n) \leq n \log_2 n + n$. The extra $+n$ term makes the base case work and gives enough room for the induction. For $n \geq 2$,

$$T(n) \leq 2 \left(\frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} \right) + n = n \log_2 \frac{n}{2} + n + n = n(\log_2 n - 1) + 2n = n \log_2 n + n.$$

The base case is $T(1) = 1 \leq 1 \log_2 1 + 1 = 1$.

Common pitfalls.

- Do not assume the result for n itself. Assume it only for the smaller input used by the recurrence.
- Check the base case. Induction without a base case is incomplete.
- The constant in the proposed bound matters. Sometimes n^2 is the right order, but the constant must be large enough for the induction to work.
- Sometimes the cleanest induction claim needs a correcting additive constant or lower-order term, such as proving $T(n) \leq cn \log n - d$ instead of only $T(n) \leq cn \log n$. For instance, a recurrence with a positive base cost may need the $-d$ term to make the algebra close exactly.

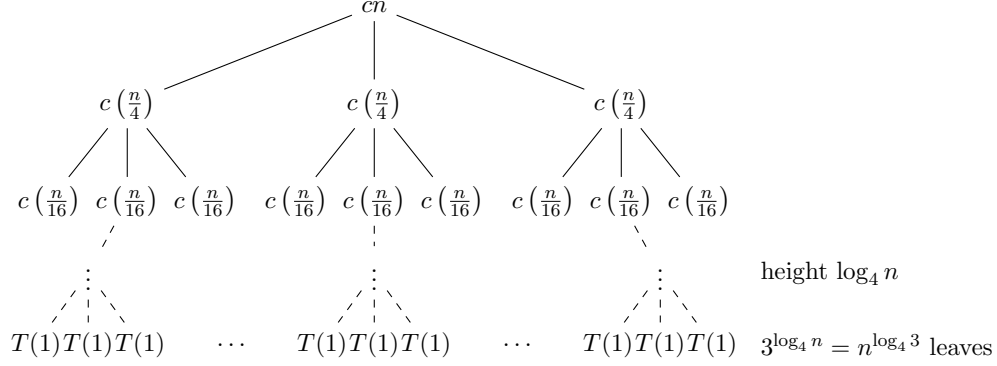
Problem 6: Recurrence Trees

Core idea. A recurrence tree organizes recursive work by levels. Count the total work at level i , then sum over all levels.

How to read a tree.

- The number of children gives the number of recursive subproblems.
- The size labels give the shrink factor.
- The work written inside a node gives the nonrecursive work at that node.

Example tree. Read the tree first, then translate it into a recurrence.



Example of correct reasoning. The tree has three recursive children, each of size $\frac{n}{4}$, and each node of size m contributes work cm . Thus it corresponds to

$$T(n) = 3T\left(\frac{n}{4}\right) + cn.$$

Now compute the work by levels:

level	number of nodes	size per node	total work
0	1	n	cn
1	3	$\frac{n}{4}$	$3c\left(\frac{n}{4}\right) = cn\left(\frac{3}{4}\right)$
2	3^2	$\frac{n}{4^2}$	$3^2 c\left(\frac{n}{4^2}\right) = cn\left(\frac{3}{4}\right)^2$
i	3^i	$\frac{n}{4^i}$	$3^i c\left(\frac{n}{4^i}\right) = cn\left(\frac{3}{4}\right)^i$

The tree stops when $\frac{n}{4^i} = 1$, so the height is $h = \log_4 n$. There are $3^h = 3^{\log_4 n} = n^{\log_4 3}$ leaves. Therefore the nonrecursive work above the leaves is bounded by

$$\sum_{i=0}^{h-1} cn \left(\frac{3}{4}\right)^i \leq cn \sum_{i=0}^{\infty} \left(\frac{3}{4}\right)^i = cn \cdot \frac{1}{1 - \frac{3}{4}} = 4cn.$$

If the base case has cost d , then the leaves contribute

$$d \cdot 3^h = d n^{\log_4 3}.$$

Since $\log_4 3 < 1$, the leaf cost is smaller than the linear internal work, so the total running time is $\mathcal{O}(n)$.

Master theorem check. For the same recurrence,

$$T(n) = 3T\left(\frac{n}{4}\right) + cn,$$

we have $a = 3$, $b = 4$, and

$$n^{\log_b a} = n^{\log_4 3}.$$

The function cn is polynomially larger than $n^{\log_4 3}$ because

$$cn \in \Omega\left(n^{\log_4 3 + \varepsilon}\right) \quad \text{for any } 0 < \varepsilon \leq 1 - \log_4 3.$$

For example, taking $\varepsilon = (1 - \log_4 3)/2$ works. Thus this is Master theorem case 3. The regularity condition holds because

$$3c \left(\frac{n}{4}\right) = \frac{3}{4}cn \leq \alpha cn$$

for some constant $\alpha < 1$. Therefore $T(n) \in \mathcal{O}(n)$.

Common pitfalls.

- Do not confuse the branching factor with the shrink factor.
- The tree height is based on the shrink factor, not the branching factor. In the example above the height is $\log_4 n$, not $\log_3 n$.
- Do not multiply the root cost by the number of levels unless each level has the same total cost.

Problem 7: Random Permutations and Indicator Variables

Core idea. Indicator random variables turn counting questions into sums of simple 0-1 variables. Linearity of expectation says

$$\mathbb{E}[X_1 + \dots + X_n] = \mathbb{E}[X_1] + \dots + \mathbb{E}[X_n],$$

even if the variables are not independent.

Example of correct reasoning. Let $A[1..n]$ be a uniform random permutation, and count how many positions contain one of the values $1, 2, \dots, k$, where $1 \leq k \leq n$. Define

$$X_i = \begin{cases} 1, & \text{if } A[i] \leq k, \\ 0, & \text{otherwise.} \end{cases}$$

Then $X = \sum_{i=1}^n X_i$ counts the positions containing one of the first k values. For a fixed i , the value in position i is equally likely to be any of $1, \dots, n$, so $\Pr[A[i] \leq k] = \frac{k}{n}$ and $\mathbb{E}[X_i] = \frac{k}{n}$. Therefore

$$\mathbb{E}[X] = \sum_{i=1}^n \frac{k}{n} = k.$$

Common pitfalls.

- Do not try to list all permutations unless n is tiny.
- Independence is not required for linearity of expectation.
- The random variables in permutation problems often depend on each other, but linearity of expectation still applies.

Problem 8: Probabilistic Algorithm Analysis

Core idea. Analyze the number of loop iterations separately from the random choices made inside those loops. If a random variable is a sum of many small contributions, indicators and linearity of expectation are usually the cleanest method.

Example pseudocode.

```
COUNTSIXES( $n$ )
 $c \leftarrow 0$ 
for  $i \leftarrow 1$  to  $n$  do
    for  $j \leftarrow 1$  to  $i$  do
         $r \leftarrow$  a uniformly random integer from  $\{1, \dots, 6\}$ 
        if  $r = 6$  then  $c \leftarrow c + 1$ 
    return  $c$ 
```

Example of correct running-time reasoning. The nested loops execute

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Thus the running time is $\mathcal{O}(n^2)$.

Example of correct expectation reasoning. If each iteration rolls a fair six-sided die and increments c only on a 6, define $X_{ij} = 1$ if the roll on iteration (i, j) is a 6 and 0 otherwise. Then $\mathbb{E}[X_{ij}] = \frac{1}{6}$, and

$$\mathbb{E}[c] = \sum_{i=1}^n \sum_{j=1}^i \mathbb{E}[X_{ij}] = \frac{1}{6} \cdot \frac{n(n+1)}{2} = \frac{n(n+1)}{12}.$$

Common pitfalls.

- Do not count the inner loop as n iterations for every i if its range depends on i .
- Do not confuse expected output with running time. The loop count here is fixed; the returned value is random.
- Do not replace a triangular sum by n^2 when an exact expectation is requested.